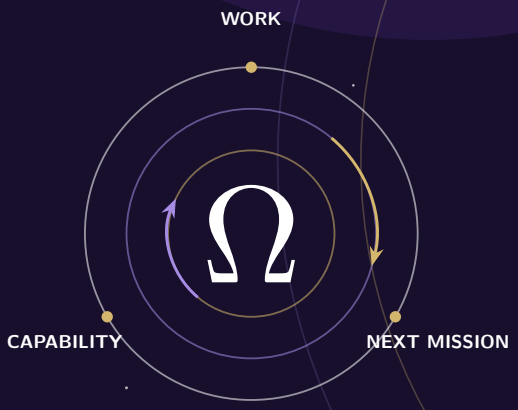
MONTREAL.AI

RESEARCH EDITION 01

AGI ALPHA FLYWHEEL

From Completed Work to Compounding Capability



KEEP THE MEANS. RE-EARN THE PROOF.

SUCCESSOR Ω HARNESS

ACTIVATE. EVOLVE. SCALE.

Powered by TRANSFORMATIONAL INTELLIGENCE ∞
and the applicable $AGIALPHA utility.

Vincent Boucher

Montréal, Québec, Canada / 24 October 2026

An executed finite software study. Token and settlement interfaces are source-inspected only.

Abstract

An autonomous work loop can finish many tasks while retaining little that makes later work easier. Conversely, a growing archive can increase checking and maintenance costs without expanding useful capability. We study the narrower, falsifiable question: can accepted work leave executable means that improve coverage of fresh requirements after the costs of acquisition, validation, failure and retention are charged? We implement a finite multi-generation source-summary experiment through the existing AGI ALPHA `createWorkbench(adapters)` source seam, preserving its frozen input, result-pack and empty-authority semantics. The acquired object is a bounded selector program and its scope, rather than a previously generated answer. The protocol fixes changing source-graph requirements, information access, four comparison arms, cumulative work budgets and stopping rules before confirmation. The strong conventional arm can cache, retrieve and adapt the same artifacts. We test removal and replacement, stale dependencies, interruption and recovery, and increasing finite load. Across 1,152 terminal evaluations, the flywheel qualified 271 of 288 tasks at 2,017,944 aggregate structural-work units, exactly tying the conventional cache; the frozen initial method qualified 58, and the authored expert workflow qualified all 288 at 2,134,383 units. In the registered archive-clearing counterfactual, retention added six qualified tasks but cost 37,184 extra units. A later regime favored rebuilding at equal quality. Sixteen local fault checks passed, while an annotation-omission challenge retained a locally accepted false conclusion. These findings concern exact structural coverage under supplied annotations and declared work accounting. They establish neither factual understanding, open-ended self-improvement, production qualification nor commercial growth. The companion retains raw evidence, failed cases, intervention disclosures, frozen source identities and an immediately usable browser demonstration.

The research question

Can a bounded system turn accepted work into reusable means of production that improve what it can accomplish next, under the same cumulative budget and current authority?

Keywords. Capability retention; recurring operation; transformational intelligence; source-grounded summaries; negative transfer; full-cost evaluation; successor qualification.

At a glance

Observation	What the evidence establishes
The loop ran	192 source-workbench epoch packs; six generations per stream; one primary launch and zero manual phase triggers.
Means were retained	Exact prior-generation recipes were used on 138 fresh terminal tasks; 134 qualified. Every output was checked again.
Reuse had a price	Registered archive intervention: 36/36 with retention versus 30/36 without it, at 37,184 additional work units.
The baseline mattered	Conventional cache tied exactly. Authored expert workflow achieved 288/288 versus 271/288 for the flywheel.
The model could fail	Omitted prerequisite: locally accepted conclusion, invalid under the separately supplied true graph.

Artifact identity. This edition uses released Alpha source `787bc547f8c4` (v1.41.1). The complete source identifier, unchanged upstream dependency hashes, frozen protocol, raw runs and claim docket are included in the research companion. The paper is a new edition; it preserves the earlier Harness detector study and its negative results. A source experiment, a browser demonstration, a maintained native installation and a persistent service have different evidence scopes.

1 Completed work should leave usable means

A completed deliverable is an outcome. A retained capability is a means of producing outcomes under a stated scope. The difference becomes observable only on subsequent work. A script that summarizes yesterday's document is not evidence that tomorrow's document can be handled. A reusable selection procedure can become such evidence if it is actually used on fresh requirements, checked against their current criteria, and charged for its full acquisition and upkeep.

The AGI ALPHA project already contains successor orchestration, cycle commands, journals, workbench families, recovery mechanisms and research scheduling. Adding another general framework would obscure the concrete question. We therefore use an existing released execution seam and supply a small research adapter and a finite stream driver. The program selects each subsequent permitted mission from its frozen stream without a person triggering every phase. It records what it retains, what it rejects and why it stops. Its execution is local to the process running the companion.

Our mission family is deliberately interpretable. A document supplies exact clauses, importance weights and an acyclic prerequisite graph. The requested artifact is a short extractive summary. A conclusion earns structural credit only when its stated prerequisites are present. The system can construct and retain a method for selecting such clauses, reuse that method on a differently ordered document, or reject it when a changed dependency structure makes it unsuitable. The problem is useful as an inspectable prerequisite-preservation instrument. The experiment does not claim that supplied annotations are correct or that structural coverage measures human understanding.

1.1 Contribution and non-substitution

The contribution is an executable assessment of recurring retention: an exact-source implementation, a fixed multi-generation experiment, a capability cost ledger, causal removal/replacement tests, and operational failure receipts. It does not introduce program libraries or automatic curricula as new ideas. DreamCoder learns reusable symbolic abstractions and search policies; Voyager combines an automatic curriculum with a library of executable skills [10, 17]. Our question is narrower: what survives after construction, checking, changed requirements and failed reuse are charged to a complete finite stream?

Cost-aware agent evaluation and held-out tasks are necessary to distinguish a useful system from an expensive benchmark success [11]. Harmful reuse is a form of negative transfer with an established literature [18]. We add a concrete research artifact and a transparent record of its boundaries. An equally capable conventional cache is a required comparison, not an adversary engineered to lose.

The earlier SUCCESSOR Ω HARNESS detector edition remains authoritative for its own experiment [4]. Its enterprise and conventional cached arms each used 32,675 counted rational calls per 96-request scenario; the direct expert used 32,583. All arms had equal primary-budget coverage. Five model-truth violations remained in the amended challenge evidence. In the hidden-gain case, the reported interval $[98, 102]$ excludes a compatible world's true mean of 92. No claim in the present study repairs that failure. Here the changing object is a retained method across recurring source-summary missions, not a detector presented under new names.

2 What growth would mean

We define the claim before discussing the result. Capability, operation and commerce require different observations.

Definition 1 (Capability growth under a budget)**.** *Let $\mathcal{T}$ be a declared set of fresh tasks, A_g the permitted executable assets retained at generation g, and B a cumulative resource ceiling. Qualified coverage is*

$$Q_g(B) = \sum_{t \in \mathcal{T}} w_t \, \mathbf{1}\{\textit{current requirements of } t \textit{ are met before } B\}.$$

The cost prefix includes obtaining A_g, unsuccessful candidates, current checks and retention work. A claim of capability growth requires higher qualified coverage, or a better quality–cost frontier, relative to a declared starting system under comparable conditions.

In this study $w_t = 1$ and qualification is exact structural optimality under the supplied graph, within the clause cap, with no unsupported selected claim. It is stricter than merely producing some valid text. A

cheap but incomplete method may remain reliable while qualifying fewer tasks. More bytes, methods, agents or nominal generations are not the endpoint.

Operational growth concerns throughput, latency, reliability, recovery and required oversight as the admitted load changes. It requires observation over the stated execution period. A finite load test describes that workload and host; it does not establish a continuously operated enterprise.

Commercial growth requires real demand, accepted delivery and attributable economic outcomes. Synthetic requirements, a token address and a zero-funded work draft supply none of those observations. Commercial growth is outside this study's measured scope.

2.1 The capability balance sheet

For an asset a over horizon T, let $C_{\text{build}}(a)$ be its construction cost, $C_{\text{qual}}(a)$ its initial checking cost, and $C_{\text{upkeep}}(a,T)$ the sum of retrieval, compatibility checks, storage, requalification and retirement costs. Let $\mathcal{U}_a$ be the compatible subsequent uses, and let $s_t(a)$ denote the saving relative to the strongest eligible alternative on task t, including unsuccessful use. Then

$$V_T(a) = \sum_{t \in \mathcal{U}_a} s_t(a) - C_{\text{build}}(a) - C_{\text{qual}}(a) - C_{\text{upkeep}}(a,T).$$

This is a bookkeeping identity, not a forecast of future value. Costs can be vectors; adding money, seconds and human minutes without an explicit conversion would be meaningless. Our primary ledger uses a declared structural-work proxy and reports timing, storage and unknown labor separately.

Proposition 1 (Finite reuse crossover)**.** *If an asset saves a constant $s > 0$ per compatible use, costs $c \geq 0$ to obtain and qualify, incurs maintenance $m \geq 0$ per use, and has no other effects, then it has strictly positive net saving after n uses exactly when $n(s-m) > c$. If $s \leq m$, no positive finite reuse count suffices.*

Proof. Substitution gives $V_n = n(s-m) - c$; solving $V_n > 0$ yields the claim. The assumptions exclude regime changes, failures and opportunity costs, which the empirical ledger must restore. □

The proposition explains why a useful artifact can still be a poor investment at short horizons, and why an indefinitely growing archive is not free. When an identical conventional method is available at the same cost, there is no special advantage in renaming it an autonomous capability. The measurement must locate the actual changed action, saved construction or avoided failure.

2.2 Retention is conditional

A retained asset contains an executable selector, an identity, a source/dependency binding, a validity scope and historical evidence. Its previous acceptance does not establish current suitability. A new task may change the graph, the permitted summary size or the evaluator. The system must be able to keep the incumbent, select a simpler method, rebuild, abstain, wait or stop. Retirement preserves the historical record while removing an asset from automatic reuse. These are useful outcomes of the flywheel.

3 Transformational Intelligence as inspectable work

Transformational Intelligence distinguishes the authorized purpose, its current formulation and the instrument used to judge it [7]. The six Out-stage papers develop complementary operations: changing required information, reasoning representations, feasible designs, strategic commitments, completion boundaries and the means of further production [1–3, 5, 6, 8]. Here those labels name observable steps in one program, rather than seven independently intelligent agents.

The foundations also constrain the claims. Out-Learn charges calibration and reports a crossover; cheap use cannot erase its initial cost. Out-Think shows that an obsolete internal representation can be more expensive to preserve than replace. Out-Execute requires current acceptance even when a reusable boundary contract saves recomputation. Out-Evolve asks whether retained means expand a budget-conditioned frontier. We retain those distinctions instead of treating every stored object as progress.

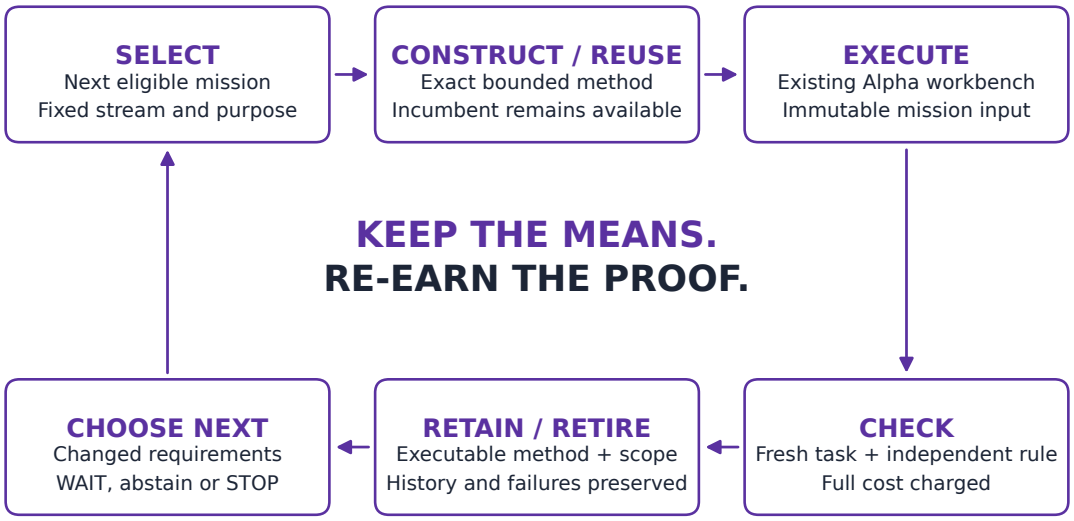


Figure 1: The bounded mission-to-capability loop. A fresh task consumes a permitted executable asset; current checking precedes retention. Historical evidence crosses generations. Active proof and authority do not. The figure describes the study's control structure, not additional deployed services.

Stage	Operation	Inspectable output
Identify	Route the next fixed mission; inspect retained methods and maintenance state.	Mission identity, incumbent, budget and scope.
Out-Learn	Compare methods on development documents; propose reuse or construction.	Development diagnostics and proposed method.
Out-Think	Instantiate and hash the exact bounded candidate descriptor.	Source-bound program identity.
Out-Design	Check that candidate on two fresh qualification documents and compute admission.	Qualification results, decision and full charges.
Out-Strategise	Apply the decision: retain the incumbent or update the active archive.	Retained set and retirement record.
Out-Execute	Execute six terminal tasks through the workbench adapter and current oracle.	Selected clause IDs, checks and charges.
Out-Evolve	Charge retention/tombstones and return state for the next fixed mission.	Result pack and next-generation handoff.

Table 1: The seven-stage vocabulary is tied to operations. Detailed stage traces identify which calls were actually executed.

4 The existing execution path and its scope

4.1 Source inspection before construction

We inspected Alpha main `787bc547f8c44854931b316d7286b8bc9f9c8bba`, tree `fddf8462e4508462e2bb0a591a1e940aa9da97ee`, and the current owner coordination in issue #4850 [13]. The repository's supported operator is `alpha-agent`. The preserved backend `AgentManager` and its cycle timer are research infrastructure; their presence does not prove a qualified service is running.

The native `DiscoveryLoop` and `CampaignExperiment` provide real campaign phases and durable journal operations. However, the policy study explicitly fixes `archive_access='none'`. Running successive policy studies would therefore not demonstrate inherited executable assets. The native `TransformationLibrary` supports archive/review/retirement operations, but routing returns no execution or spending authority.

The signed native journal's initialization also creates a local identity key and API token. This task excludes new credentials, so the study did not initialize that runtime. The primary executed seam is the released, credential-free JavaScript `createWorkbench(adapters)` interface. Its original modules are vendored unchanged and hashed. The research adapter supplies a bounded source-summary method; the finite experiment driver sequences predetermined missions and persists research receipts. It is not a replacement scheduler or an installed native capability.

4.2 What the released workbench supplies

The workbench validates a normalized mission, constructs a bounded job description, binds the mission and source hash, executes the installed adapter and hashes its output artifacts. Its result records `independentProof:false`, `admission:HOLD`, empty authority and `settlement:NOT_CONNECTED`. Import inspection returns historical information with empty active proof and authority. Our implementation preserves these fields.

The task-specific research driver is limited to the fixed stream, its closed selector artifacts and local evidence files. It cannot send email, acquire funds, create an account, call a model provider or modify production source. A person launches a finite run once. The process then selects and executes the subsequent eligible work according to the frozen policy; reaching a limit produces a terminal record. Closing a browser tab stops that tab's fresh computation. The packaged Node process runs only while its host process remains alive.

4.3 Proof and authority remain separate

Object	Included evidence	Limitation
WORLD	Source clauses, supplied weights, dependency edges and regime definitions.	Annotation correctness and user value are premises.
POLICY	Fixed method grammar, routing, retention and stopping rules.	No arbitrary generated code or evolving objective.
PROOF	Exact-task result, independent structural check, source and protocol identities.	Local experimental acceptance; no institutional certification.
AUTHORITY	Finite local-computation scope, current limits and expiry checks.	No production grant, funding, identity or settlement permission.

Table 2: Four separately inspectable objects. A successor's historical evidence does not become an active grant.

Candidate programs are inert closed-language data. They cannot modify the evaluator or authority fields through the supported interface. The source and criterion hashes are frozen before confirmation. The same research workspace and team control the environment; this is interface separation, not a claim of an adversarially isolated laboratory. A second same-session reviewer and an independently written checker provide additional scrutiny with that custody limit stated.

5 Prospective experiment

The prospective protocol fixes eight complete paired streams, with seeds 80317, 190061, 271003, 380041, 491027, 580031, 690037 and 790051. Each stream contains six successive regimes. Each regime supplies two development documents, two candidate-qualification documents and six untouched terminal documents. Thus each arm has 36 terminal opportunities per stream, 288 in aggregate; the four arms jointly provide 1,152 terminal opportunities. The task bytes were generated and hashed before primary execution. Development uses excluded seed 101 and is preserved separately.

Epoch	Clauses	Output cap	New requirement
G0	5	3	Independent clauses, importance order: the simple incumbent can win.
G1	6	2	Permuted independent clauses: selection by importance can help.
G2	7	3	Changed importance and order with random acyclic prerequisites.
G3	8	2	A new prerequisite graph and tighter output cap.
G4	9	3	Independent clauses return: a compatible specialization can avoid search.
G5	10	1	New graph with one output clause: empirical suitability may fail.

Table 3: Frozen changing requirements. These are six structural regimes, not six names for the same input. Their distribution is authored and synthetic.

The closed method grammar contains `lead-order`, `weighted-without-dependencies`, `audit-after-draft` and `dependency-graph-before-compression`. A retained recipe selects one supplied primitive and binds its source, identifier, creation epoch and historical scope. It is an executable configuration of a supplied algorithm, not invented general-purpose code. Candidates cannot change the grammar, evaluator or permit.

The flywheel first inspects a compatible retained method on both development documents. The shipped five-method comparison runs and is charged in either case. If the retained method is supported and exactly optimal on both, it keeps the original identity and skips the additional instrumented construction pass. Otherwise it executes the four allowed methods and chooses the greatest supported development weight, breaking ties by declared work. A candidate must be supported and exactly optimal on both fresh qualification documents and cannot reduce their total score relative to the incumbent. Failure retains the incumbent. Every terminal output is still checked; qualification on two documents does not establish a universal scope.

The conventional cache uses precisely this policy, with the same information, grammar and retention limit. Equal logical outcomes are therefore expected by construction. This is an explicit equivalence control, not an independently discovered algorithmic comparison. The frozen arm keeps the initial lead-order method. The expert workflow inspects the visible graph and uses importance ordering when there are no edges and exact graph selection otherwise. It receives no future labels. Its human construction labor was not measured.

Budget and retirement. The primary ceiling is 500,000 standardized structural-work units per complete stream, charged cumulatively from the first mission. Unexecuted terminal tasks count zero. At most two typed methods remain active; deterministic least-recently-used retirement preserves tombstones and history. A changed loaded-source binding prevents reuse. The finite task manifest, budget, local permit expiry and explicit STOP are stopping conditions. WAIT means there is no currently due eligible task; it does not create a persistent service.

For a document with n clauses, e supplied edges and cap k, the shipped five-method comparison receives the frozen tariff

$$n2^n + \left(\sum_{i=0}^{k} \binom{n}{i}\right)(n+e+1) + 5n + 2e.$$

The remaining rules charge instrumented selector operations, source reads, routing, selected KiB serialization/hash blocks and one SDK invocation unit. This explicit tariff is not an instruction trace. Complete process wall and CPU time and written bytes are measured separately. Budget prefixes at 25,000, 50,000, 100,000 and 200,000 units are descriptive projections of the registered execution order, not separately optimized or rerun policies at those budgets.

Freeze and amendment. The source binding is `d03406c0df742f269ca3d21e1ca973c2504e08d4303417dda4510652e71eb6db`; the task-byte hash is `be1a82132f0e0d6f24971142ab6ddea0f0772e72d3bf183b2aed34e1ede51fe8`. The unexecuted checkpoint was saved before the primary run and recorded in issue #4850, comment 6387744548. One checker-schema adapter changed after the freeze and before execution; the original bytes and explicit

amendment are preserved. The numerical reference core, receipt checker, candidate source, evaluation criteria and task bytes were unchanged. No primary outputs had been observed. Later amendments, if any, must be separately identified.

5.1 Qualification and the opportunity frontier

For a source document with clause set V, weight $w_i > 0$, prerequisites D_i and cap k, a selected set $S \subseteq V$ is supported if every transitive prerequisite of every selected clause is also in S. The declared optimum is

$$u^*(V, D, w, k) = \max_{S \subseteq V : |S| \le k,\, \mathrm{cl}(S) \subseteq S} \sum_{i \in S} w_i.$$

A fresh mission qualifies if its output obeys the cap, has no unsupported selected clause, and reaches u^*. Equality is exact; no floating-point tolerance or subjective model judge is used. An independent exhaustive enumeration checks this finite optimum. The checker verifies the selected clause identities against the bound source document; the source clauses and provenance remain inspectable.

This instrument offers a meaningful but limited guarantee: selected conclusions retain the prerequisites represented in the supplied graph. It cannot detect an omitted dependency absent from that graph. The separately disclosed annotation challenge makes that failure visible. A successful structural check cannot be promoted into a truth claim.

5.2 Matched arms and usable controls

The inheritance arm and conventional cache have the same starting grammar, task information, available tools and budget. Both may retain and adapt accepted selector artifacts under the same compatibility policy. The no-new-capability arm isolates the effect of permitting acquisition, but is not the strongest comparison. The direct expert is a prewritten conventional procedure, not a measured human-subject experiment; its authored routing rule and construction charges are disclosed. No arm may inspect confirmation labels while choosing a candidate.

Removing a selector and replacing its descriptor test two different claims. Removal asks whether the available selector matters. Replacement constructs a separately digested descriptor with the same method and a changed creation-epoch field, then executes the same selector code. It tests dependence on that identity/history metadata; it does not establish independent rediscovery. The separate whole-stream archive-clearing intervention measures the effect of retaining means across missions.

5.3 Accounting and uncertainty

The primary budget uses the explicitly defined standardized structural-work ledger. It is a transparent proxy for work, not CPU instructions, energy or money. It charges candidate generation and evaluation, invoked comparator work, fresh checking, retrieval, selected serialization/hash blocks, retention decisions and maintenance under that declared tariff. One workbench-invocation unit does not measure every compiler or cryptographic instruction. Failures remain charged. Timings and storage are measured separately; authoring effort, human labor valuation and production maintenance remain unmeasured.

The complete finite stream is the comparison unit. Documents within a stream share methods and costs, so treating every clause or document as an independent replication would overstate precision. We report paired stream outcomes and their range. With only a small authored population, narrow numerical intervals do not establish generalization to unknown real documents. The confirmation material is prospectively held apart from candidate development, but all input generation and custody remain within the same team. The adaptive-data-analysis literature explains why repeated reuse of a holdout would require additional control [9].

6 Results: what accumulated, and what did not

6.1 The loop executed; the strongest baseline did not lose

All eight paired streams completed their six fixed epochs. The primary run produced 192 workbench result packs and 1,152 terminal observations after one launch, with no human selection or phase trigger inside the

run. No stream exhausted its 500,000-unit ceiling. The separate Python checker reconciled every terminal optimum, selected source binding, capability identity, result-pack hash and 140,388 cost events. Both the original frozen checker adapter and its disclosed pre-primary schema amendment passed; this is separate implementation evidence within the same research custody.

Workflow	Qualified / 288	Total work units	Units / qualified
Flywheel	271	2,017,944	7,446.29
Conventional cache	271	2,017,944	7,446.29
No acquired capability	58	1,412,176	24,347.86
Authored expert	288	2,134,383	7,411.05

Table 4: Primary result at the same cumulative ceiling per stream. Total work includes acquisition and checking from the first mission. The final column is a descriptive equal-task-value ratio, not monetary productivity. Expert authoring labor remains unknown.

The flywheel achieved 94.1% structural qualification versus 20.1% for the fixed initial method. That acquisition comparison does not establish a unique advantage: the equally equipped conventional cache tied on every logical output and charge, as expected from its identical policy. The expert workflow qualified all tasks and led every registered lower-budget prefix. Its greater aggregate work than the flywheel purchased 17 additional qualifying outcomes; its aggregate units per qualifying task were slightly lower. Neither a universal cost saving nor superiority over the strongest conventional workflow is supported.

Stream	Flywheel	Cache	Frozen	Expert	Flywheel work
0	36	36	8	36	316,616
1	33	33	7	36	229,675
2	32	32	8	36	229,416
3	30	30	6	36	235,775
4	35	35	8	36	229,442
5	36	36	8	36	316,196
6	35	35	6	36	230,646
7	34	34	7	36	230,178

Table 5: All paired complete-stream results, each out of 36 opportunities. The observed population is these eight authored streams.

The paired flywheel-minus-frozen difference averaged 26.625 qualified tasks per stream, with range $[24, 29]$. The flywheel-minus-expert difference averaged -2.125, with range $[-6, 0]$; the cache difference was identically zero. Flywheel-minus-expert work differences ranged from $-37,209$ to $+49,146$. These finite paired results give the observed variation without implying document-level independent samples or population confidence. Figure 2 shows the budget-conditioned evidence, including the lower-budget expert lead.

6.2 A method crosses a generation; its proof does not

In stream 0, G0 retains the simple lead selector. G1 acquires a weighted method for permuted independent clauses. G2 acquires exact prerequisite-aware selection. G3 reuses that graph method under a new graph and tighter cap; G4 returns to the weighted method from G1; G5 reuses the G2 graph method at cap one. The original recipe digests survive these uses, and fresh checking remains mandatory (Figure 3). This is a changed executable action set rather than a repeated answer. The methods themselves come from the supplied grammar.

Across the flywheel's 48 epoch decisions, 16 admitted a research recipe, 31 retained the incumbent and one rejected a descendant. Those are decision events, not 16 distinct algorithms: four distinct content identities occurred across the streams. A capability attached to a row is not necessarily inherited; its creation epoch can equal the current epoch. Resolving identity to creation epoch yields 138 actual later-generation terminal uses, of which 134 qualified. Imported active proof and authority remain empty throughout.

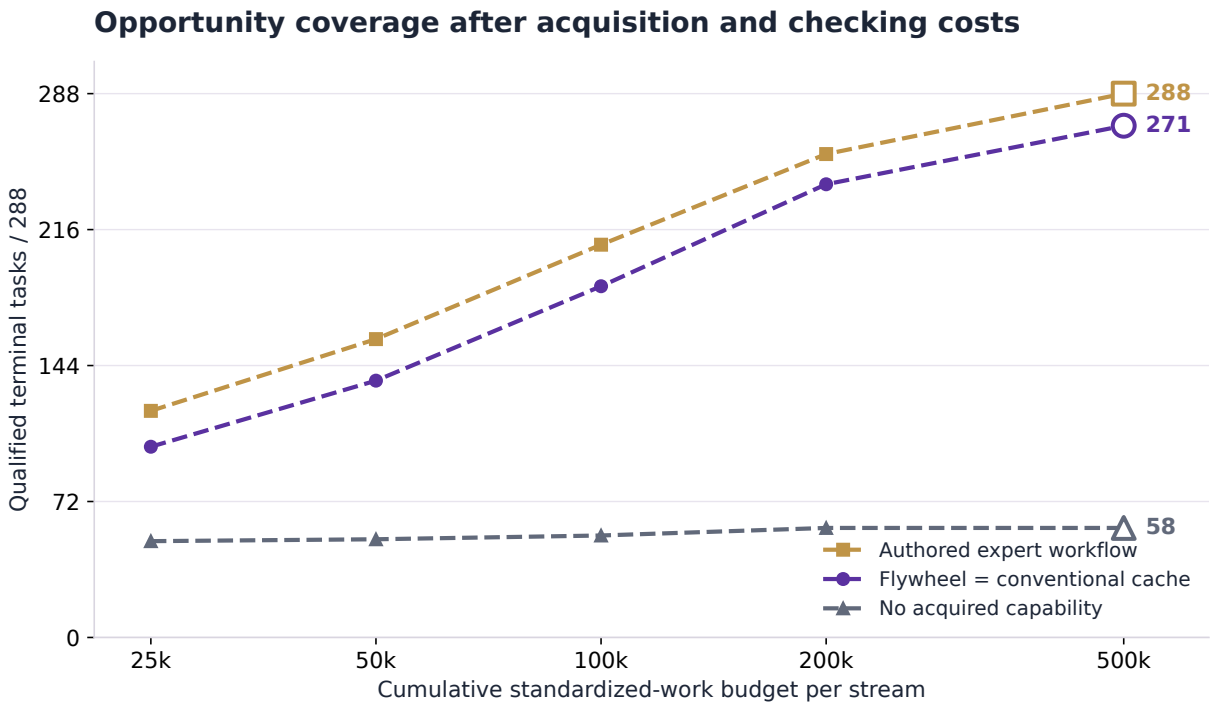


Figure 2: Opportunity coverage across the fixed budget accounting prefixes. Only the 500,000-unit endpoint is the registered primary budget; intermediate points project the same recorded execution order. They are not freshly optimized policies or repeated experiments. The conventional cache is exactly coincident with the flywheel.

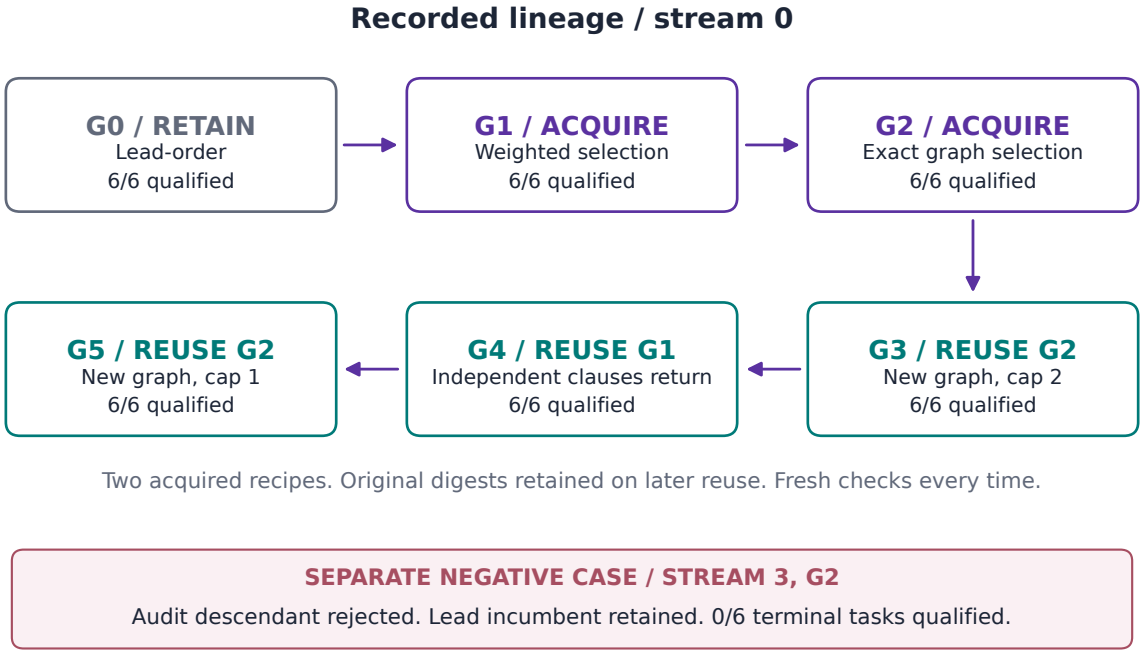


Figure 3: An actual six-generation successful lineage, with a separately identified negative case from another stream. Keeping the initial method and rejecting a descendant are observable decisions, not missing records.

6.3 Removal, replacement and a genuine persistence intervention

The registered component-removal test forces the initial lead selector on stream 0's same 36 terminal tasks, yielding 8 qualifying results. A newly digested, semantically equivalent selector replacement yields 36. On the 18 tasks that actually reuse an earlier-epoch recipe, removal yields 2/18 and replacement 18/18. Thus the selector matters, while changing this descriptor's identity and creation metadata does not change its output. The replacement uses the same selector code; it is not independent invention. These separately charged tests do not reproduce acquisition economics and must not be read as the archive effect.

The stronger persistence test clears the active archive before each epoch and allows the same flywheel policy to construct and qualify candidates again. It qualifies 30/36 tasks at 279,432 units, compared with 36/36 at 316,616 units with retention. Persistence therefore adds six qualified outcomes at 37,184 additional units under the registered ceiling. It does not reduce total cost. All qualification and failed reconstruction work remains in both totals.

Epoch	Retained quality	Cleared quality	Retained work	Cleared work
G0	6/6	6/6	4,318	4,318
G1	6/6	6/6	7,163	7,163
G2	6/6	6/6	32,893	32,891
G3	6/6	0/6	43,884	34,098
G4	6/6	6/6	59,795	72,061
G5	6/6	6/6	168,563	128,901

Table 6: Stream 0 archive intervention, with incremental epoch costs. G4 shows a 12,266-unit reuse saving at equal quality. G5 reverses the preference: rebuilding saves 39,662 units at the same 6/6 observed quality. Neither local result is silently generalized to all future tasks.

At G3, clearing the graph capability exposes a failed qualification and leaves a poor incumbent. At G4, restoring the weighted capability avoids the additional instrumented construction pass; the shared comparison still runs and is charged. At G5, retained graph selection remains adequate on development documents, so the policy keeps its expensive exact method; rebuilding chooses the cheaper audit method and reaches the same six terminal optima. The current policy tests whether an incumbent is adequate, not whether a cheaper newly available method exists. That specific retention rule creates the measured liability. Exploring alternatives periodically may help, but that would be a new policy and a new experiment.

6.4 Failures and the full declared cost

Of the flywheel's 17 nonqualifying terminal outputs, 11 are supported but suboptimal audit summaries. The rejected descendant in stream 3, G2 leaves six nonqualifying lead summaries: three supported but suboptimal and three with missing prerequisites. Passing two candidate-qualification documents did not establish a universally suitable scope. No failed output is counted as qualified.

The omitted-annotation challenge is a different failure. Its submitted graph omits the prerequisite of a high-weight conclusion. The method selects that conclusion at cap one and obtains local supported score 20; under the separately supplied true graph it is unsupported and scores zero. This authored counterexample is excluded from the structural primary endpoint and retained as evidence that a correct checker cannot repair a false world model.

For the flywheel, 1,053,126 units—52.2% of total work—are terminal validation; construction/search costs 406,382, candidate qualification 350,670 and terminal execution 206,861. Routing, retention, maintenance and SDK invocation contribute the remaining 905. Reuse avoids some construction but does not eliminate current validation. A claim based only on terminal execution would omit most attributable work under this declared tariff.

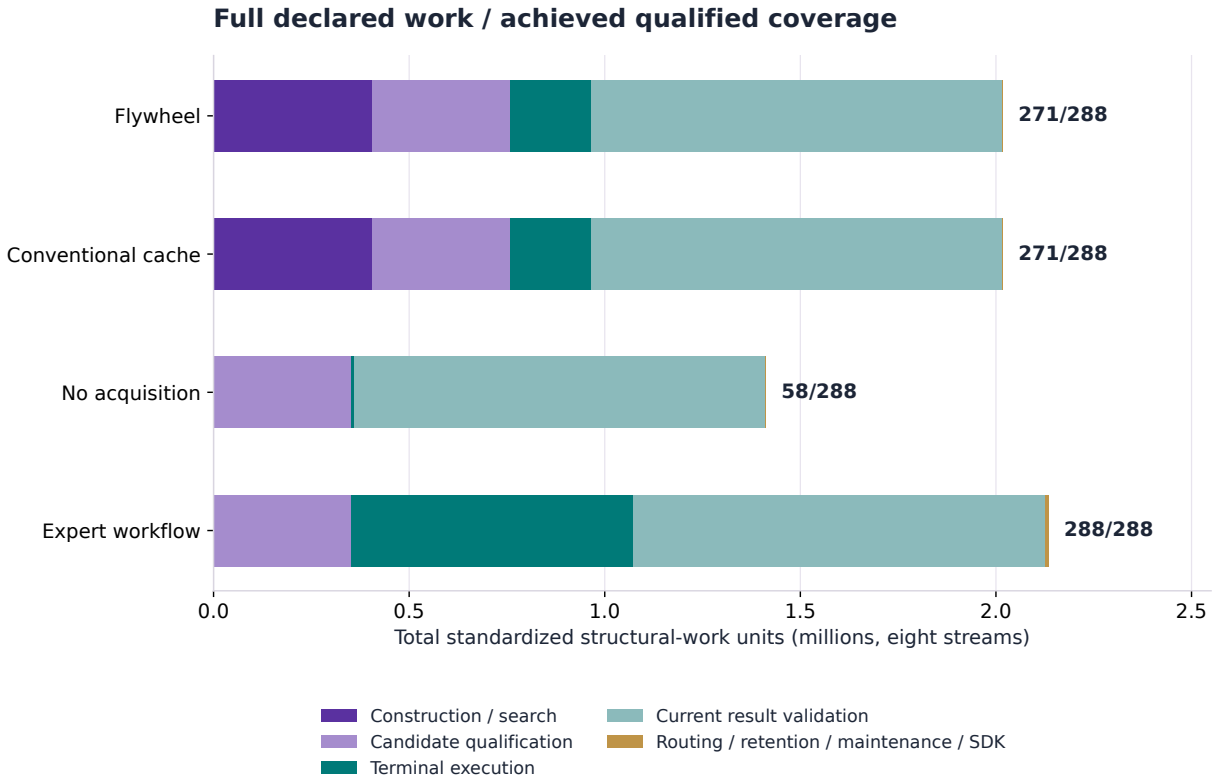


Figure 4: Every category in the frozen structural-work ledger. Small routing, retention, maintenance and invocation charges are included even when visually thin. The proxy does not price human/model authoring, energy or money; complete process timing and storage are reported separately.

7 Recovery, load and the cost of operating

7.1 Committed work survives; verification still costs work

All 16 declared local fault cases produced their expected decisions. A new process recovered a committed result; a lost acknowledgement and an exact duplicate returned the prior identity with no new mission result. Each of those three checks recomputed deterministic semantics at 10,721 verification units. Replaying a rejected descendant consumed a further 137,669 verification units and left it rejected. Thus these four checks added 169,832 verification units; returning a saved receipt is not free and does not establish exactly-once computation.

The changed-dependency test copies the actual frozen source closure, alters a module and starts a new process; it does not merely change a label. The rollback test executes the exact earlier accepted recipe on a new requirement with empty proof and authority. No native deployment or production rollback is claimed. The rehashed false report is particularly instructive: content-addressed integrity does not establish semantic correctness. Current input binding and independent checking are necessary even when the hashes agree.

7.2 Finite load: higher throughput, higher latency, unchanged quality

The load experiment executes 16 epoch tasks, containing 96 terminal outputs, under each of one, two and four actual Node processes. Each setting was measured once on the same host. Throughput includes process startup; task latency begins when a worker starts that task and omits earlier queue waiting.

Four processes deliver 2.33 times the single-process epoch throughput in this batch, while median task latency and total measured CPU increase. Quality does not improve. The same inputs and failures explain the identical 66/96 score; parallelism increases completion rate, not capability. No extrapolation beyond four processes, the finite load or this host is warranted.

Injected condition	Observed boundary
Restart, lost reply, exact duplicate	Same committed identity; no fresh result; explicit semantic recomputation.
Same identity, changed input	`DUPLICATE_REQUEST_CONFLICT`.
Changed dependency bytes	New process refuses old source binding before execution.
Corrupt receipt	Native pack integrity check refuses it.
Rehashed false report	Native integrity check accepts its hashes; added semantic check rejects its content.
Self-consistent task swap / forged recipe	Exact input or capability binding refuses it.
Budget exhausted / permit expired	`STOP_BUDGET` after two charged units / expiry refusal.
STOP before commitment	Cancellation retains 2,467 units of partial work; retry computes the uncommitted mission again.
No eligible work	WAIT until the declared due condition, then READY.
Earlier accepted recipe / rejected descendant	Prior recipe actually executes fresh requirements; rejected descendant remains rejected after restart.

Table 7: Observed local failure boundaries, grouped for readability; the raw suite contains 16 cases. "Authority" expiry here means the finite local research permit. It is not a production grant.

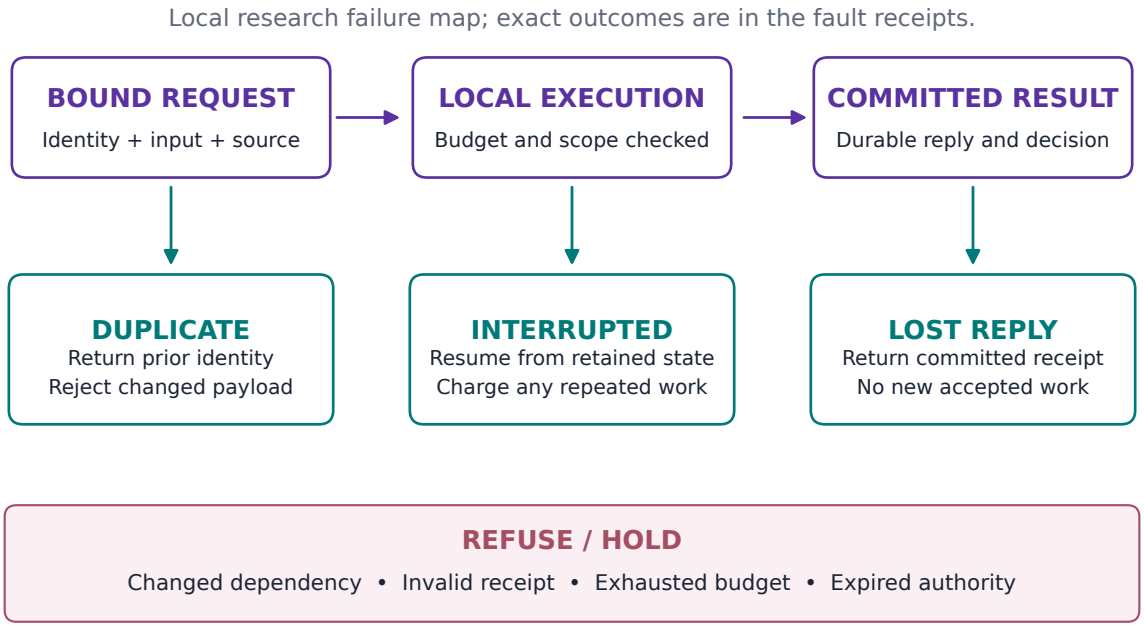


Figure 5: The local recovery map. Refusal, reuse and recomputation are distinct outcomes. The separate Python operational review checks receipts and arithmetic; it does not independently re-inject the same faults.

Processes	Epochs/s	p50 ms	p95 ms	CPU s	Qualified
1	17.74	47.20	71.68	0.979	66/96
2	26.34	59.18	106.15	1.259	66/96
4	41.29	77.62	99.77	1.469	66/96

Table 8: Bounded load observations. Each setting emits approximately 2.155 MB of result bytes and requires zero within-batch human interventions. Thirty nonqualifying outputs remain in every setting. This is not sustained service throughput or a production service-level measurement.

7.3 Storage, maintenance and intervention accounting

The primary experiment used 15.528 seconds of wall time and 15.164 seconds of process CPU (11.756 user, 3.408 system), and wrote 47,020,876 bytes before its summary file. This includes the four arms, checking and evidence serialization. A clean source-only reproduction used 14.913 seconds wall time and reproduced all 1,152 normalized scientific observations, 140,388 logical charges and 192 generation decisions exactly. Fresh elapsed times, creation dates and their derived pack hashes are intentionally not expected to match. Running the advertised resume command over all 192 saved reproduction packs issued zero fresh mission results, but performed 192 verification re-executions at 12,938,483 additional proxy units and 7.124 seconds of summed verification wall time. This expensive reconciliation is not charged as if it were part of the uninterrupted primary run, nor concealed as free recovery.

The active inventory never exceeded two recipes or 789 canonical serialized bytes. That small inventory must not be confused with total retained evidence: one complete stream/arm accumulated as much as 1,345,850 bytes of result packs, and the primary ledger alone occupies 17,427,710 bytes. The flywheel incurred 610 routing, 144 retention and 103 maintenance proxy units in addition to its execution and checking work. Capacity-driven retirement was implemented but never reached in the primary streams; there were zero primary retirement events. Dependency mismatch was exercised separately. An indefinitely retained audit history remains unbounded without a separate archival policy.

The operator ledger separates one user commission, research-team task selection, source inspection, setup, debugging, method review, freeze approval, launches, fault injection and publication preparation. The primary stream has one launch and zero manual phase interventions. This does not assign zero cost to the research or setup: their human time, authoring-model inference and economic valuation were not measured. All excluded development failures and corrections remain available, including the pre-freeze change that first made compatible reuse skip unnecessary reconstruction.

Durable identities and retained replies have a long history in distributed systems; RIFL is one example [12]. Our local tests do not establish distributed exactly-once semantics. A reply lost after a committed result can be returned from its retained record. A crash before commitment can require recomputation, whose cost must remain visible. The distinction between duplicate computation and duplicate accepted work is material. External effects are excluded from this experiment.

The no-intervention claim is limited to the executed stream. Mission-family choice, protocol design, implementation, setup, debugging, initial launch and research review were performed before or around that run. The intervention ledger records those categories and unknown durations. Automated phase progression is useful evidence; it is not evidence that all research, deployment or oversight became autonomous.

8 Four repositories and applicable token utility

The four-product integration is inspected, not declared complete. The audit binds Alpha to `787bc547`, Manager to `c5958eae`, Jobs v0 to `2e234ab9`, and Fleet Linux to `45759fb0`. Exact file paths, Git blob identifiers and SHA-256 hashes are in the companion audit [13–16]. Static source admissions do not automatically admit a newer Alpha release.

Product	Verified interface and possible role	Executed in this study
Alpha	Released workbench source extension; pure source selector and result-pack inspection.	Source SDK computation.
Manager	Discovery work-request inspection/admission on its USDC6 route.	Source inspection only.
Jobs v0	Discovery work-request inspection/admission on its AGIALPHA18 route.	Source inspection only.
Fleet Linux	Bounded project/receipt interfaces for permitted execution and returned work.	Source inspection only.

Table 9: Connected interface awareness is distinct from installed integration or settlement. The source request format preserves zero-cap, draft and disconnected states.

The five Alpha contracts serve distinct potential functions. `AscensionAccess` checks registration, admission, expiry and current ownership; `NovaSeed` binds a capsule, plan and evidence to a seed and optional parent. Neither makes the attached content true. `AscensionRiskOracle` provides stake, risk review and validator-capacity functions. `AscensionJobMarket` provides bounty escrow, bidding, award, result-bound validation,

settlement or refund. `AscensionMark` provides reversible funding lots, bloom and plan-constrained treasury routing. Table 10 records the relevant names without implying execution.

Contract	Relevant source functions
`AscensionAccess`	`register`, `setAdmission`, `owns`, `admitted`, `eligible`
`NovaSeed`	`seal`, `plan`, `creatorOf`
`AscensionRiskOracle`	`deposit`, `withdraw`, `openReview`, `attest`, `green`, `lockForJob`, `releaseFromJob`
`AscensionJobMarket`	`post`, `postFromSeed`, `bid`, `award`, `submit`, `validate`, `timeout`, `claimRefund`
`AscensionMark`	`open`, `quote`, `buy`, `sell`, `transferLots`, `bloom`, `route`, `collect`, `close`, `reclaim`

Table 10: Five-contract source map. No listed function was called by this experiment. The audit is not a security audit or a current deployed-byte verification.

Applicable $AGIALPHA utility would be escrow, agent collateral, validator stake, payouts and reserves where those exact contracts and admissible workflows are used. Manager's USDC6 path remains separate from Jobs' AGIALPHA18 path; no conversion or shared funding is inferred. This study is unsigned, unfunded and disconnected from settlement. It performs no local EVM experiment and no live financial action. Token issuance, supply, scarcity or price is not a productive-value endpoint.

The deployment guide contains historical point-in-time observations, including an October 18 source/block binding. We did not refresh them through RPC. The original Harness and Fleet action restrictions remain with their existing owners. The paper, a passing structural check and a new source commit cannot clear them.

9 A demonstration that exposes its evidence

Open the self-contained `demo/index.html` in a current browser. The opening view presents one clear choice: replay the recorded evidence or run fresh local computation. English and French controls, keyboard operation, responsive layouts and reduced-motion behavior make the experiment approachable without an account or command line. The source package also provides the same finite loop as a Node command for complete reproduction.

The recorded mode walks through actual generation decisions and reveals the full cost/coverage comparison, exact capability identities, failed qualifications, archive-clearing tradeoff and recorded failure/load receipts. It preserves the expert lead and conventional tie. Its recovery view is labeled as recorded evidence; clicking a replay control does not repeat the original operating test.

The fresh mode verifies embedded module hashes, then imports the exact packaged kernel and unchanged upstream modules through local browser module URLs. It can execute a fresh six-generation finite stream, stop it and export its checkpoint. A smaller structural instrument permits direct inspection of a changed input. These are fresh local computations, distinct from the primary measurements and excluded from confirmatory totals. The source hash and local execution scope are visible in the evidence drawer. The page needs no external fetch, wallet, credential, model API or hosted service.

The visual journey is intentionally short: start the stream, see the first retained method, inspect a later reuse or rejection, reveal cumulative cost, inspect interruption/recovery, compare the strongest baseline and examine the successor's inherited objects. The first action is obvious; additional evidence remains available without forcing a reader to understand repository or contract vocabulary.

Recorded replay and fresh computation have distinct labels. A replay is a view of preserved measurements. The fresh structural instrument executes a shipped bounded method locally on permitted inputs. Neither mode runs a service after its executing process closes. For a persistent operating installation, the existing runtime owner must supply the maintained source, current scope, configured journal and explicit service lifetime; this manuscript does not provision that service.

10 Limitations and claims that survive them

The input population is authored, small and structurally narrow. We supply weights and dependencies, and we supply the selector language. The system does not invent general software primitives, understand arbitrary documents or discover a market. The strong conventional workflow is allowed the same reuse because otherwise an apparent gain would confound memory access with intelligence.

The resource ledger makes attribution inspectable but does not price every real cost. Wall time varies with host load, process startup, garbage collection and I/O. Counted logical work treats unlike operations through a declared proxy. Human labor, research design, model inference used to author the system, energy and long-run maintenance are not fully observed. Therefore the study can support a conditional structural coverage result and a finite operational result; it cannot establish total economic superiority.

Local file hashes bind bytes and detect ordinary corruption. They do not authenticate an external custodian or prove a measurement true. The authors control the frozen source and task generator; same-session review is not institutional independence. The public code permits inspection and reproduction, not retroactive blindness. An external evaluation on independently collected tasks and measured human outcomes would be a distinct experiment.

Load observations are bounded samples, with no continuous production history, customer service-level agreement or observed demand. Unfunded drafts and source-inspected contracts remain disconnected. The scope deliberately leaves the original Harness/Fleet review and action decisions untouched.

The durable standard

Prove that the loop runs. Test whether retained means improve later work. Keep the negative cases. Renew the evidence and authority for each new scope.

11 Conclusion

The finite autonomous loop ran: one launch produced successive source-bound missions, retained executable recipes, reused them on fresh requirements, rejected a descendant and stopped at its declared horizon. Acquisition improved structural coverage over the fixed initial selector, and the archive intervention isolated a real contribution from earlier work. Capability did not improve monotonically or uniquely: the strongest cache tied by design, the authored expert led coverage at every registered budget prefix, and retaining a costly incumbent lost to rebuilding in a later regime. These are conditional capability results within a supplied grammar and graph model, not evidence of open-ended intelligence, sustained production growth or economic superiority. The reusable result is a means of acting under a declared scope, together with evidence of its costs and failures. A successful flywheel must retain useful means, reject liabilities and remain able to stop. That is a testable basis for the ambition behind TRANSFORMATIONAL INTELLIGENCE ∞ and SUCCESSOR Ω HARNESS.

FIND YOUR ALPHA. BUILD YOUR SUCCESSOR Ω.

Author and implementation disclosure

Vincent Boucher, MONTREAL.AI, commissioned this research and defined its authorized scope. AI-assisted engineering and research workers prepared the implementation, audit, verification and manuscript in one coordinated session. The independent-checker and review roles denote separate implementations or work roles within that session, not independent institutions. No external peer-review acceptance is claimed.

Artifact availability and license

The complete companion includes the authoritative PDF, compilable LaTeX, bibliography, editable SVG/PDF figures, unchanged upstream source closure, frozen protocols and manifests, raw results, failed attempts, intervention ledger, evidence docket, reproduction scripts and publication handoff. Apache-2.0 applies to new source; original notices remain on vendored files. Final archive hashes and durable-copy

verification accompany the delivered edition. Website publication status is reported separately from artifact completion.

References

[1] Vincent Boucher. Out-design: Creating the freedom to transform, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Design.tex. MONTREAL.AI revised research manuscript, October 21.

[2] Vincent Boucher. Out-evolve: Building the means of discovery and production, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Evolve.pdf. MONTREAL.AI revised research manuscript, October 20.

[3] Vincent Boucher. Out-execute: Constructing the conditions of completion, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Execute.tex. MONTREAL.AI revised research manuscript, October 21.

[4] Vincent Boucher. SUCCESSOR Ω harness: Make every mission build what comes next, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/papers/successor-omega-harness-detector-v1.pdf. Preferred 28-page detector research edition, October 24.

[5] Vincent Boucher. Out-learn: Change what you need to know, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Learn.tex. Constructed Experiments and Sharp Calibration Guarantees for Reusable Learning; revised October 21.

[6] Vincent Boucher. Out-strategise: Transformational intelligence for constructing the conditions of success, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Strategise.tex. MONTREAL.AI research manuscript, October 20.

[7] Vincent Boucher. Transformational intelligence: Learning to change the game, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Transformational_Intelligence.tex. MONTREAL.AI research manuscript, October 2026.

[8] Vincent Boucher. Out-think: Transformational intelligence for inventing the space of reasoning, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/blob/787bc547f8c44854931b316d7286b8bc9f9c8bba/docs/Out_Think.tex. MONTREAL.AI research manuscript, October 21.

[9] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse, 2015. URL https://arxiv.org/abs/1506.02629.

[10] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sable-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning, 2020. URL https://arxiv.org/abs/2006.08381.

[11] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. Ai agents that matter, 2024. URL https://arxiv.org/abs/2407.01502.

[12] Collin Lee, Seo Jin Park, Ankita Kejriwal, Satoshi Matsushita, and John Ousterhout. Implementing linearizability at large scale and low latency. In *Proceedings of the 25th ACM Symposium on Operating Systems Principles*, 2015. doi: 10.1145/2815400.2815416. URL https://sigops.org/s/conferences/sosp/2015/current/abstracts.html.

[13] MONTREAL.AI. AGI Alpha Agent v0: released source and operator documentation, 2026. URL https://github.com/MontrealAI/AGI-Alpha-Agent-v0/tree/787bc547f8c44854931b316d7286b8bc9f9c8bba. Version 1.41.1; exact commit 787bc547f8c44854931b316d7286b8bc9f9c8bba; inspected October 24.

[14] MONTREAL.AI. AGI Fleet Linux: source interface snapshot, 2026. URL https://github.com/MontrealAI/AGI-Fleet-Linux/tree/45759fb096faf8988f58ae5ea71248d0f35a97c7. Exact commit 45759fb096faf8988f58ae5ea71248d0f35a97c7; inspected October 24.

[15] MONTREAL.AI. AGI Jobs v0: source interface snapshot, 2026. URL https://github.com/MontrealAI/AGIJobsv0/tree/2e234ab9b8f44eda0a2525b3ce75fc87ee7a4c7f. Exact commit 2e234ab9b8f44eda0a2525b3ce75fc87ee7a4c7f; inspected October 24.

[16] MONTREAL.AI. AGI Jobs Manager: source interface snapshot, 2026. URL https://github.com/MontrealAI/AGIJobManager/tree/c5958eae872458c2f3cac48fea0b4bbfadc6ac6c. Exact commit c5958eae872458c2f3cac48fea0b4bbfadc6ac6c; inspected October 24.

[17] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291.

[18] Wen Zhang, Lingfei Deng, Lei Zhang, and Dongrui Wu. A survey on negative transfer, 2020. URL https://arxiv.org/abs/2009.00909. Revised August 2021; published in IEEE/CAA Journal of Automatica Sinica, 2022.

A Reproduction and evidence docket

The tested computational environment is Linux x86_64, Node 24.22.1 and Python 3.12.14. The research runner uses Node built-ins and included source modules; the mathematical checker uses the Python standard library. No account, model API, wallet, package download or network request is needed after obtaining the complete companion. Fresh outputs go into a new directory; existing evidence is not overwritten.

From the extracted package root:

```
node companion/run.mjs reproduce fresh-run
python3 review/independent/review_results.py \
  companion/fresh-run --output fresh-review.json
node companion/run.mjs resume fresh-run
```

The resume invocation reconciles saved immutable packs, performs additional semantic verification and grants a new finite local research permit only. It does not enable production actions. To inspect the browser demonstration, open `demo/index.html`; its embedded code and data are self-contained.

Ancillary experiments can be regenerated separately:

```
node companion/run.mjs faults fresh-faults
node companion/run.mjs load fresh-load
node companion/run.mjs ablations fresh-ablations
```

The ablation command reads the original recorded stream-0 results as its registered reference. Its outputs do not alter primary selection. It must not be represented as a new independently blinded holdout.

To rebuild the paper, use XeLaTeX/BibTeX from TeX Live 2023 with the listed packages and Latin Modern fonts, then run:

```
python3 paper/build_paper.py --output rebuilt-paper
```

The script disables shell escape, fixes the source timestamp and compiles in a new directory. The authoritative PDF remains in `paper/`. Editable SVG/PDF figures and their generator are included. Figure regeneration additionally uses pinned Matplotlib 3.10.8; compilation itself uses the supplied vector PDFs.

Claim	Primary artifact	What can be checked
C01	`companion/freeze.json`	Exact experiment source, task bytes, pre-execution status and checker chronology.
C02	`companion/results/summary.json`	Four-arm coverage, complete stream totals, budget prefixes, process timing and primary intervention counters.
C03	`companion/results/generations.json`	Accepted/rejected methods, original identity on reuse, scope and empty inherited authority.
C04	`companion/results/ledger.jsonl`	All 140,388 original logical cost events; no summation of repeated cumulative snapshots.
C05	`companion/ablations/`	Forced component removal, descriptor replacement, whole-stream archive clearing and annotation failure.
C06	`companion/faults/`	Sixteen expected boundaries, partial interrupted costs, semantic forgery and rejected-result restart.
C07	`companion/load/load.json`	Actual process counts, finite throughput, latency, CPU, quality and output bytes.
C08	`companion/analysis/`	Active recipe bytes versus complete evidence history and epoch-specific retention tradeoffs.
C09	`review/independent/`	Separate reference mathematics, receipt/accounting checks, amendments and their independence limits.
C10	`companion/reproduction-verification.json`	Exact scientific/accounting equivalence with specified nondeterministic fields excluded.

Claim	Primary artifact	What can be checked
C11	`review/browser-qa.json`	Exact demo hash and 24 real-browser checks, including fresh execution and mobile/keyboard behavior.
C12	`audit/`	Frozen four-repository/five-contract source map; no executed financial interface.
C13	`evidence/intervention-ledger-authoring.json`	Research/operator setup, corrections, launches, reviews and unmeasured labor categories.

The machine-readable `evidence/claim-docket.json` links these claims to exact artifacts and scopes. The package's hash manifest verifies individual files, while the delivery verification binds final archives and durable readback. Hashes identify bytes; they do not certify a claim's truth or authorize an action.

B Exact interfaces and inherited state

A capability artifact carries the bounded method and provenance required for the next computation. The next mission carries its own input and scope, and receives its own result pack. The retained historical result is not current proof. The workbench import returns `UNAUTHENTICATED_HISTORICAL_IMPORT`, `activeProof:[]` and `activeAuthority:[]`. The companion demonstrates those boundaries directly.

The mission stream remains a finite research driver. It uses the existing workbench's adapter interface and checkpoint callback; the separately maintained native `successor-study` operation owner retains the timed driver lane. No source in the held `enterprise.py`, original Harness or Fleet objectives is imported or reimplemented by this paper.

C Preservation of the preceding research edition

The preferred detector PDF's SHA-256 is `326762251ff24993a8fc5baeaa6316a33a1bb08ba9f5475f5c0b17a05b6b357e`. Its 28-page edition, conventional tie, expert advantage, retained timing losses and gain-error failure are preserved as prior evidence. We cite its result; we do not substitute this paper or silently update its comparator values. The new study's original data and amendments have their own versioned identities.